



Gestión de hilos

INICIAR >

Navigation icons: back, forward, search, and a menu icon.

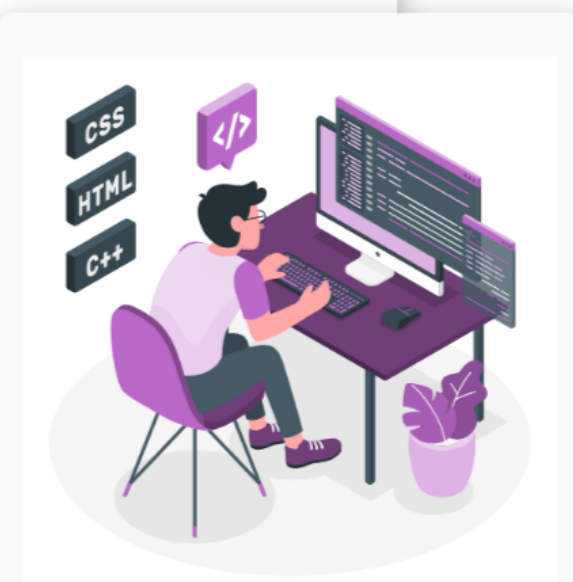


Gestión de hilos

Bienvenida

El desarrollo de hilos en tus programas te dejará beneficios como:

- La ejecución simultánea de múltiples tareas.
- Para codificarlos, *Java* proporciona dos formas sencillas para crear y controlar hilos.
- Permite sincronizar estos subprocesos con los métodos que elabores en tus proyectos.



Navigation icons: back, forward, search, and a menu icon.



Gestión de hilos

Creación y control de hilos

Java es un lenguaje que está fuertemente ligado al paradigma de la programación orientada a objetos y trata a los hilos como objetos.

Para crear un hilo tienes dos opciones:

1.- Heredar la clase *Thread*

2.- Implementar la interfaz *Runnable*

En ambos casos, es necesario sobrescribir el método *run()* con el código que se ejecutará.

Estructura de un hilo heredado de la clase *Thread*

```
Public class Hilos extends Thread {  
    @Override  
    public void run(){  
        //código  
    }  
}
```

Ejecución del hilo

```
public static void main( String args[] ) {  
    Hilos t1;  
    t1 = new Hilos();  
    t1.start();  
}
```



Gestión de hilos

Creación y control de hilos

```
public class Hilos extends Thread {  
    @Override  
    public void run() {  
        //código  
    }  
    public static void main( String args[] ) {  
        Hilos t1= new Hilos();  
        t1.start();  
    }  
}
```

Para **crear un hilo nuevo con la clase heredada *Thread***, es necesario:

1. Crear una clase nueva y heredar la clase *Thread*
2. Sobrescribir el método *run()* heredado
3. Crear una instancia de clase en el método *main*
4. Iniciar la ejecución del hilo mediante el método *start()*

Importante:

Puedes crear tantas instancias de clase como quieras.

Pero toma en cuenta que los hilos en ejecución se crean hasta invocar el método *start()*.



Gestión de hilos

Creación y control de hilos

Para llamar al **método `run()`** no debes hacerlo mediante las instancias que creaste. Cuando quieras ejecutar el hilo únicamente usa el **método `start()`**.

Observa detenidamente ambos ejemplos

Correcto

```
Public static void main ( string args [] ) {  
    Hilos t1 = new Hilos("Hilo 1");  
    t1.start ();  
}
```

Incorrecto

```
Public static void main ( string [] args) {  
    Hilos t1 = new Hilos("Hilo 1");  
    t1.run ();  
}
```



Gestión de hilos

Creación y control de hilos

Implementar la interfaz *Runnable*

Estructura de un hilo implementando la interfaz Runnable

```
public class Hilo implements Runnable {  
    @Override  
    public void run() {  
        //código  
    }  
}
```

Ejecución del hilo

```
public static void main(String[] args) {  
    Hilo hilo = new Hilo();  
    Thread t1 = new Thread(hilo);  
    t1.start();  
}
```




Gestión de hilos

Creación y control de hilos

```
public class Hilo implements Runnable {  
  
    @Override  
    public void run() {  
        //código  
    }  
  
    public static void main(String[] args) {  
        Hilo hilo = new Hilo();  
        Thread t1 = new Thread(hilo);  
        t1.start();  
    }  
}
```

Para **crear un hilo nuevo desde la implementación de la interfaz *Runnable***, es necesario:

1. Crear una clase nueva e implementar la interfaz *Runnable*.
2. Sobrescribir el método *run()*.
3. Crear un objeto de clase en el método *main*.
4. Crear un objeto de la clase *Thread* y pasar como parámetro al objeto creado en el paso anterior.
5. Iniciar la ejecución del hilo mediante el método *start()* en el objeto de la clase *Thread*.

Importante:

Ya que **Java no permite la herencia múltiple**, cuando creas un hilo por medio de la herencia, lo limitas, de modo que no puede heredar de otra clase. Sin embargo, la implementación de la interface *Runnable* puede darte otras posibilidades.



Gestión de hilos

Sincronización de hilos

Consiste en controlar su entrada hacia las secciones críticas.

Para lograr esta **sincronización**, *Java* ofrece una forma sencilla de lograr la concurrencia de acceso a los métodos.

Cada hilo puede acceder solo a un método a la vez, siempre que sea declarado con la palabra reservada **synchronized**.

- Si un hilo trata de usar este método mientras que otro lo está ocupando, será detenido hasta que el primer subproceso haya terminado.
- El método declarado mediante **synchronized** se conoce como **sección crítica**.

Observa el siguiente ejemplo:

```
public synchronized void add(int amount){  
    //código  
}
```

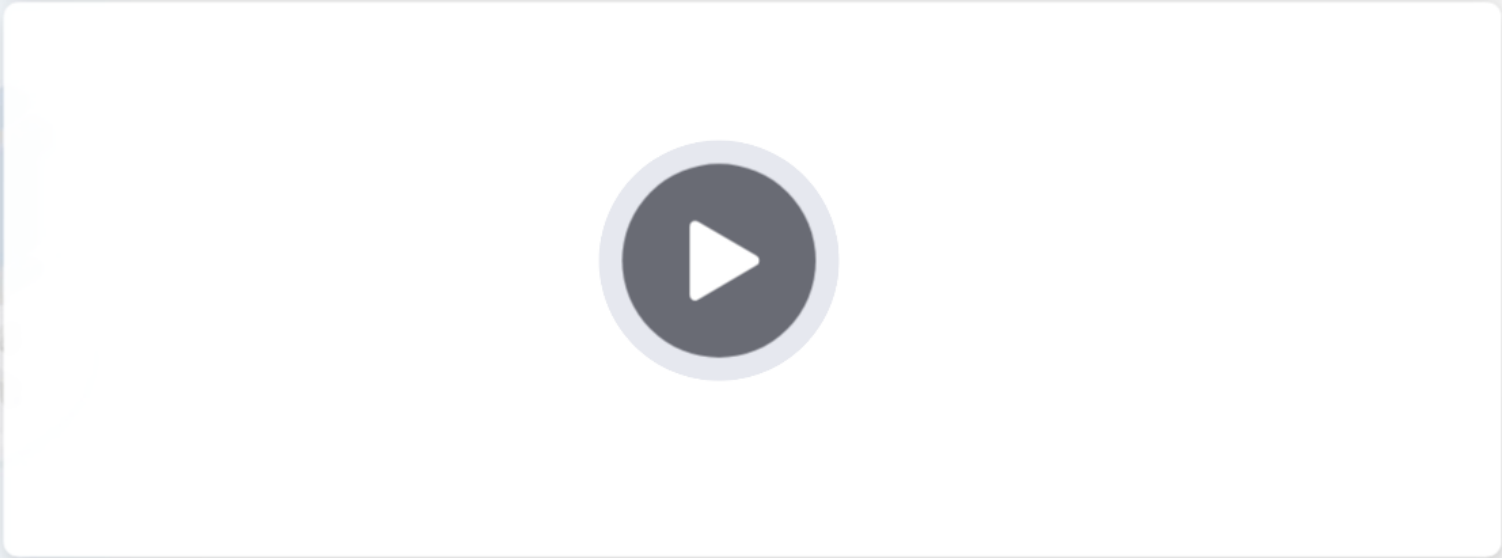


Gestión de hilos

Sincronización de hilos



A continuación, verás un video tutorial que presenta un ejemplo en el que se aplica la sincronización de hilos implementando la **interfaz Runnable**:



VIDEO: <https://youtu.be/pk94MCxfMVo?si=CKkOZk73SWKb2VE1>



Gestión de hilos

Sincronización de hilos

Java solo permite el acceso solo a una sección crítica por objeto.

- Aunque esta situación se ve afectada si se trabaja con métodos estáticos; ya que un hilo puede acceder a un método estático de la clase, solo si es afectado por la palabra **clave synchronized**.
- Mientras que otro hilo puede acceder al resto de los métodos no estáticos.

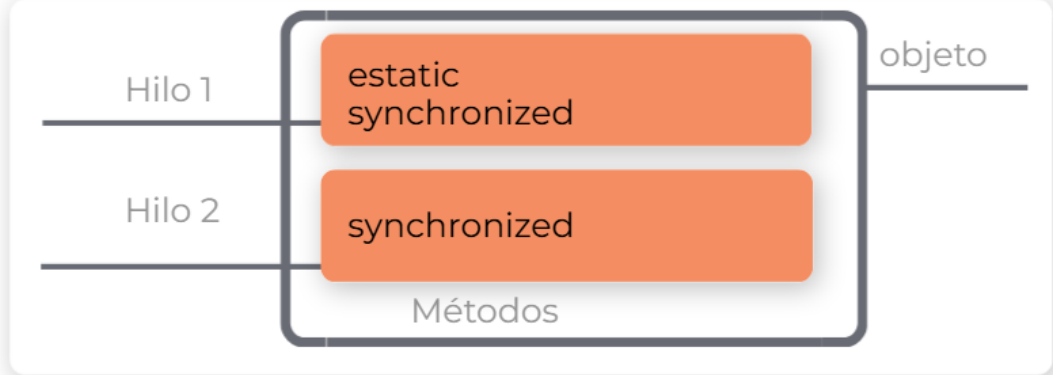


Diagram illustrating thread synchronization:

- Two threads, **Hilo 1** and **Hilo 2**, are shown on the left.
- On the right, a box labeled **Métodos** contains two orange boxes: **estatic synchronized** and **synchronized**.
- A line labeled **objeto** connects the **estatic synchronized** box to the **synchronized** box, indicating that both methods are synchronized on the same object.



Gestión de hilos

Sincronización de hilos

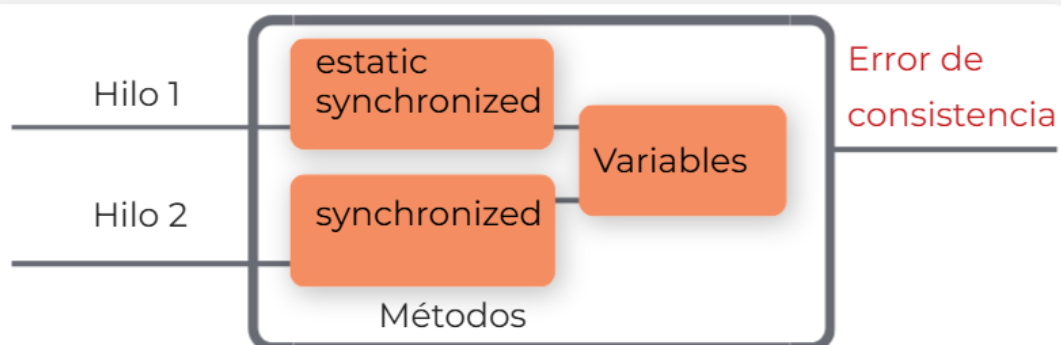


Esto podría parecer importante solo hasta que ambos métodos tengan efecto sobre las mismas variables, de modo que se generen errores de inconsistencia.

Para evitarlo, es necesario tener precauciones.



Observa el siguiente ejemplo:



Gestión de hilos

Para concluir...



- *Java* ofrece una forma muy práctica y sencilla de programar con hilos, aunque al principio te puede confundir.
- Sin embargo, te parecerá más sencillo cuando lo apliques en la codificación de tus programas.
- Considera que el tiempo que inviertas en implementar hilos en tus proyectos, repercutirá en resultados más efectivos para los usuarios finales.

